Deployment & Distribution

1 — Release Cycle Overview

CI/CD, Code Signing, Notarization and Installer

Abstract: *This paper describes the complete distribution pipeline for WhyCremisi, from build automation to final publication. It covers semantic versioning, multi-platform compilation, code signing, macOS notarization, installer creation, and GitHub Actions automation. The goal is to ensure a secure, repeatable, and verifiable release process on macOS, Windows, and Linux.*

1 — Release Cycle Overview

WhyCremisi adopts **Semantic Versioning 2.0.0** (`MAJOR.MINOR.PATCH`) with the following criteria:

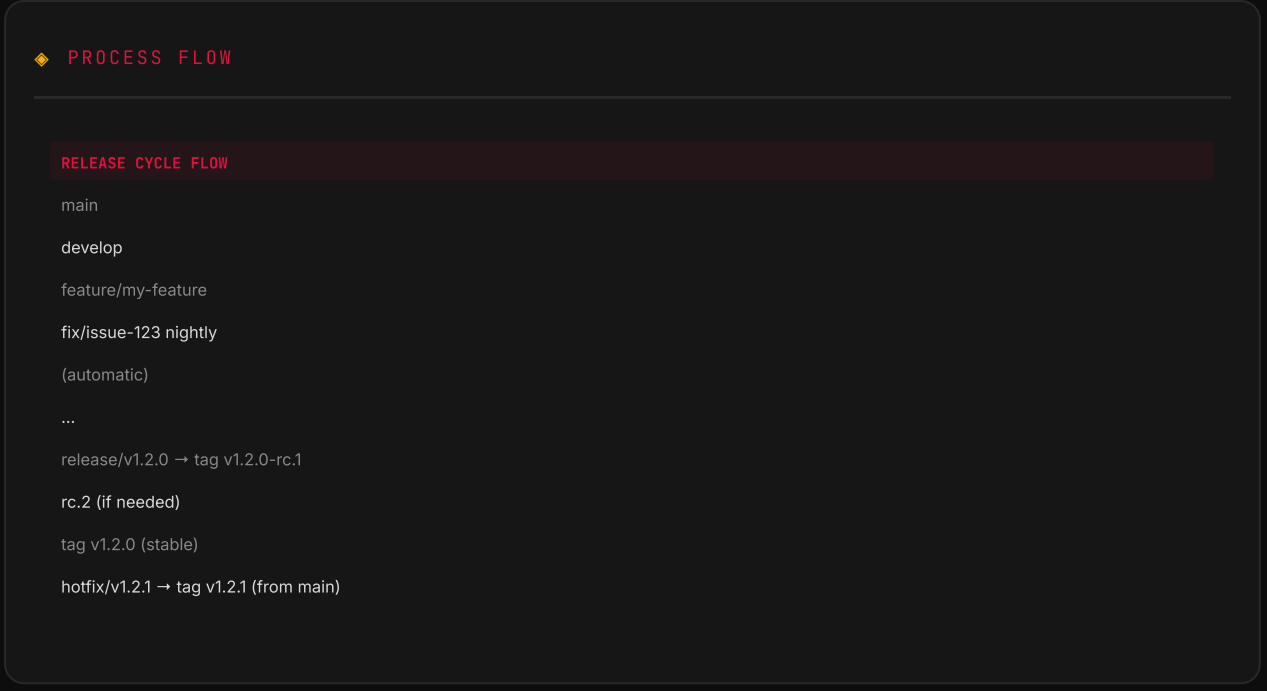
COMPONENT	WHEN TO INCREMENT	EXAMPLE
MAJOR	Breaking API/plugin change	`2.0.0`
MINOR	New feature, backward-compatible	`1.3.0`
PATCH	Bug fix, no new features	`1.2.1`

The release cycle follows four progressive stages:

◈ OVERVIEW

```
Nightly → Beta → RC → Stable
v v v v
0.x.y-dev 0.x.y-b x.y.z-rc x.y.z
```

◈ PROCESS FLOW



[NOTE] `hotfix/*` branches branch off `main` and are merged back into both `main` and `develop` to prevent regressions.

---

2 — Build Automation

WhyCremisi uses **CMake presets** to standardize builds across all three operating systems.

— 2.1 — CMAKEPRESETS.JSON

```
{
  "version": 8,
  "configurePresets": [
    {
      "name": "debug",
      "displayName": "Debug",
      "generator": "Ninja",
      "binaryDir": "${sourceDir}/build/debug",
      "cacheVariables": {
        "CMAKE_BUILD_TYPE": "Debug",
        "CMAKE_OSX_ARCHITECTURES": "x86_64;arm64"
      }
    },
    {
      "name": "release",
      "displayName": "Release",
      "generator": "Ninja",
      "binaryDir": "${sourceDir}/build/release",
      "cacheVariables": {
        "CMAKE_BUILD_TYPE": "Release",
        "CMAKE_OSX_ARCHITECTURES": "x86_64;arm64"
      }
    },
    {
      "name": "relwithdebinfo",
      "displayName": "RelWithDebInfo",
      "generator": "Ninja",
      "binaryDir": "${sourceDir}/build/relwithdebinfo",
      "cacheVariables": {
        "CMAKE_BUILD_TYPE": "RelWithDebInfo",
        "CMAKE_OSX_ARCHITECTURES": "x86_64;arm64"
      }
    }
  ],
  "buildPresets": [
    { "name": "debug",    "configurePreset": "debug" },
    { "name": "release",  "configurePreset": "release" },
    { "name": "relwithdebinfo", "configurePreset": "relwithdebinfo" }
  ]
}
```

— 2.2 — UNIVERSAL BINARY (MACOS)

`CMAKE_OSX_ARCHITECTURES` is set to `x86_64;arm64` to produce an **universal binary** compatible with both Intel and Apple Silicon:

```
cmake --preset release -G "Xcode" \
  -DCMAKE_OSX_ARCHITECTURES="x86_64;arm64" \
  -DCMAKE_XCODE_ATTRIBUTE_CODE_SIGN_IDENTITY="Developer ID Application: WhyCremisi"
cmake --build --preset release --config Release
```

— 2.3 — WINDOWS

```
cmake --preset release -G "Visual Studio 17 2022" -A x64
cmake --build --preset release --config Release
```

— 2.4 — LINUX

```
cmake --preset release -G "Ninja"
cmake --build --preset release
cpack -G DEB   # generates .deb
cpack -G RPM   # generates .rpm
```

PLATFORM	GENERATOR	PACKAGE FORMAT	ARTIFACT
macOS	Xcode	.dmg	VST3 + AU + Standalone
Windows	Visual Studio 17 2022	.exe (Inno Setup)	VST3 + AAX + Standalone
Linux	Ninja	.deb / .rpm	VST3 + Standalone

• 3 — macOS Code Signing

Code signing is mandatory for distributing audio plugins on macOS. WhyCremisi uses an **Apple Developer ID Application** certificate.

— 3.1 — CERTIFICATE & PROFILE

- **Type:** Developer ID Application (for outside Mac App Store distribution)
- **Authority:** Apple Worldwide Developer Relations Certification Authority
- **Validity:** 3 years (manual renewal)

— 3.2 — ENTITLEMENTS

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN"
  "http://www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0">
<dict>
  <key>com.apple.security.cs.disable-library-validation</key>
  <true/>
  <key>com.apple.security.cs.allow-unsigned-executable-memory</key>
  <true/>
  <key>com.apple.security.cs.allow-dyld-environment-variables</key>
  <true/>
</dict>
</plist>
```

[NOTE] `com.apple.security.cs.disable-library-validation` is **mandatory** for VST3 plugins that load third-party libraries at runtime.

— 3.3 — SIGNING COMMAND

```
# Sign the VST3 bundle
codesign --deep --force --verify --verbose \
  --options runtime \
  --sign "Developer ID Application: WhyCremisi (TEAMID)" \
  --entitlements entitlements.plist \
  WhyCremisi.vst3

# Verification
codesign --verify --verbose=4 WhyCremisi.vst3
spctl --assess --verbose=4 --type execute WhyCremisi.vst3
```

— 3.4 — HARDENED RUNTIME

The `--options runtime` flag enables **Hardened Runtime**, which applies:

- Arbitrary code execution prevention
- Heap memory protection
- System resource access control

• 4 — Windows Code Signing

Windows requires an **Extended Validation (EV) Code Signing** certificate for maximum trust.

— 4.1 — SIGNTOOL

```
signtool sign /fd SHA256 \
  /tr "http://timestamp.digicert.com" \
  /td SHA256 \
  /a \
  /v \
  WhyCremisiInstaller.exe
```

PARAMETER	DESCRIPTION
`/fd SHA256`	Digest algorithm SHA-256
`/tr <url>`	RFC 3161 timestamp server
`/td SHA256`	Timestamp digest algorithm
`/a`	Automatically select best certificate

— 4.2 — WINDOWS HARDWARE COMPATIBILITY PROGRAM

For audio drivers (if needed), WhyCremisi may undergo the **Windows Hardware Compatibility Program (WHCP)** to obtain an additional Microsoft signature that guarantees compatibility with Driver Signature Enforcement.

— 4.3 — SIGNATURE VERIFICATION

```
Get-AuthenticodeSignature -FilePath WhyCremisiInstaller.exe
```

5 — macOS Notarization

Notarization is Apple's process for verifying that software is free of malicious components.

— 5.1 — SUBMISSION WITH NOTARYTOOL

```
# Create zip package for notarization
ditto -c -k --sequesterRsrc --keepParent \
  WhyCremisi.dmg WhyCremisi-notarize.zip

# Submit to notarytool
xcrun notarytool submit \
  --apple-id "developer@whycremisi.com" \
  --team-id "TEAMID" \
  --password "@keychain:AC_PASSWORD" \
  --wait \
  WhyCremisi-notarize.zip
```

— 5.2 — UPLOADPAYLOAD.JSON

```
{
  "primaryBundleId": "com.whycremisi.plugin",
  "primaryBundleVersion": "1.2.0",
  "options": {
    "userId": "TEAMID"
  }
}
```

— 5.3 — STATUS POLLING

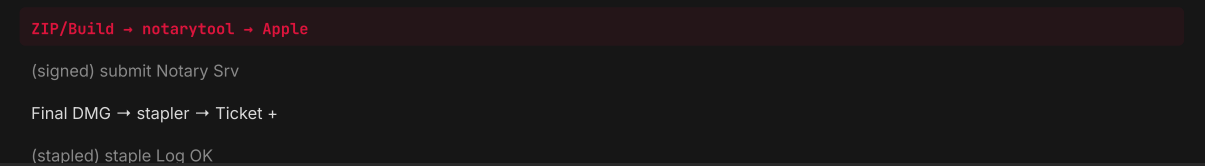
```
# Retrieve notarization status
xcrun notarytool log \
  --apple-id "developer@whycremisi.com" \
  --team-id "TEAMID" \
  --password "@keychain:AC_PASSWORD" \
  <submission-id>
```

— 5.4 — STAPLING

```
# Staple: attach the notarization ticket to the DMG
xcrun stapler staple WhyCremisi.dmg

# Verification
xcrun stapler validate WhyCremisi.dmg
spctl --assess --verbose=4 --type install WhyCremisi.dmg
```

◈ DEPLOY FLOW



6 — Installer

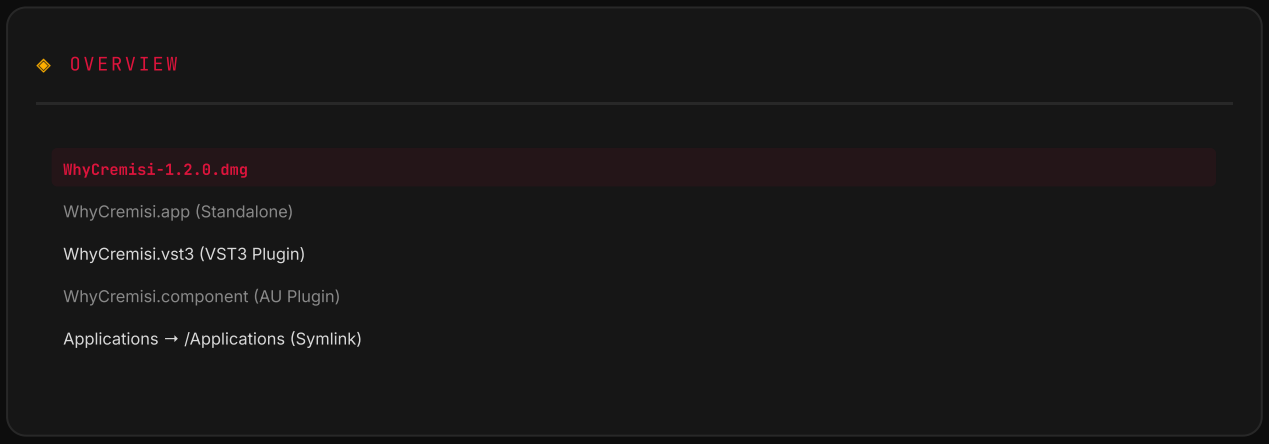
— 6.1 — MACOS: DMG

The WhyCremisi DMG includes:

- **Custom background** (WhyCremisi brand, 660×400 px)
- **/Applications symlink** for drag-and-drop install
- **EULA** displayed on volume mount
- **Component plist** for VST3, AU and Standalone

```
# DMG creation with create-dmg (open-source tool)
create-dmg \
  --volname "WhyCremisi 1.2.0" \
  --background "installer-bg.png" \
  --window-pos 200 120 \
  --window-size 660 400 \
  --icon-size 100 \
  --icon "WhyCremisi.app" 180 170 \
  --icon "WhyCremisi.vst3" 360 170 \
  --icon "WhyCremisi.component" 540 170 \
  --app-drop-link 400 170 \
  --eula "LICENSE.txt" \
  --no-internet-enable \
  "WhyCremisi-1.2.0.dmg" \
  "dist_output/"
```

DMG Structure:



— 6.2 — WINDOWS: INNO SETUP

Inno Setup script for the Windows installer:

```
[Setup]
AppName=WhyCremisi
AppVersion=1.2.0
DefaultDirName={commonpf}\WhyCremisi
DefaultGroupName=WhyCremisi
UninstallDisplayIcon={app}\WhyCremisi.exe
Compression=lzma2
SolidCompression=yes
OutputDir=dist
OutputBaseFilename=WhyCremisi-1.2.0-Setup
SignTool=MySignTool

[Components]
Name: VST3; Description: VST3 Plugin (64-bit); Types: full custom
Name: AAX; Description: AAX Plugin (Pro Tools); Types: full custom
Name: Standalone; Description: Standalone Application; Types: full custom

[Files]
Source: "build\Release\WhyCremisi.vst3"; DestDir: "{commonpf}\Common Files\VST3"; Components: VST3
Source: "build\Release\WhyCremisi.aaxplugin"; DestDir: "{commonpf}\Avid\Audio\Plug-Ins"; Components: A
Source: "build\Release\WhyCremisi.exe"; DestDir: "{app}"; Components: Standalone

[Run]
Filename: "{app}\WhyCremisi.exe"; Description: "Launch WhyCremisi"; Flags: postinstall nowait skipifs
```

— 6.3 — LINUX: .DEB / .RPM

```
# Debian/Ubuntu
cpack -G DEB
# Generates: WhyCremisi-1.2.0-Linux.deb

# Fedora/RHEL
cpack -G RPM
# Generates: WhyCremisi-1.2.0-Linux.rpm
```

PLATFORM	INSTALLER	VST3 PATH
macOS	.dmg	`/Library/Audio/Plug-Ins/VST3/`
Windows	.exe (Inno Setup)	`%COMMONPROGRAMFILES%\VST3\`
Linux	.deb / .rpm	`~/.vst3/` or `/usr/lib/vst3/`

• 7 — Distribution Channels

— 7.1 — DIRECT DOWNLOADS

The website (`whycremisi.com/download`) hosts signed packages with published SHA-256 checksums. Each release ships:

◈ OVERVIEW

WhyCremisi-1.2.0-macOS.dmg

WhyCremisi-1.2.0-Windows-Setup.exe

WhyCremisi-1.2.0-Linux.deb

WhyCremisi-1.2.0-Linux.rpm

WhyCremisi-1.2.0-SHA256SUMS.txt → checksum file

WhyCremisi-1.2.0-SHA256SUMS.txt.sig → GPG signature

— 7.2 — AUTO-UPDATE

PLATFORM	FRAMEWORK	UPDATE FORMAT
macOS	Sparkle	`.appcast.xml` + signed DMG
Windows	Squirrel	`.nupkg` + Release EXE

[NOTE] Sparkle provides an XML appcast signed with an EdDSA key. Cryptographic verification happens client-side before applying any update.

— 7.3 — LICENSE KEY VALIDATION

WhyCremisi integrates an offline/online validation system:

```
sequenceDiagram
    User—»Plugin: Enters license key
    Plugin—»Validation Server: POST /validate (key, hw-id)
    Validation Server--»Plugin: JWT token (30-day validity)
    Plugin—»Plugin: Stores token in Keychain/Registry
    Note over Plugin: On each launch verify JWT signature + expiry
    alt Token expired or invalid
        Plugin—»User: Show activation screen
    end
```

— 7.4 — TELEMETRY (OPT-IN)

Crash reporting uses **Crashpad (macOS/Windows)** with submission to a private endpoint. The user must explicitly consent during installation or via the settings menu.

8 — Release Automation

— 8.1 — GITHUB ACTIONS WORKFLOW

```yaml
# .github/workflows/release.yml
name: Release Pipeline

on:
  push:
    tags:
      - 'v*'

jobs:
  build:
    strategy:
      matrix:
        os: [macos-13, windows-2022, ubuntu-22.04]
    runs-on: ${{ matrix.os }}
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: '3.11'

      - name: Configure CMake
        run: cmake --preset release

      - name: Build
        run: cmake --build --preset release

      - name: Code Sign (macOS)
        if: runner.os == 'macOS'
        run: |
          echo "${{ secrets.MAC_CERTIFICATE }}" | base64 --decode > cert.p12
          security create-keychain -p temp build.keychain
          security import cert.p12 -k build.keychain -P "${{ secrets.MAC_CERT_PASSWORD }}"
          security set-key-partition-list -S apple-tool:,apple: -s -k temp build.keychain
          cmake --build --preset release --target sign

      - name: Notarize (macOS)
        if: runner.os == 'macOS'
        run: |
          xcrun notarytool submit \
            --apple-id "${{ secrets.APPLE_ID }}" \
            --team-id "${{ secrets.TEAM_ID }}" \
            --password "${{ secrets.APPLE_APP_PASSWORD }}" \
            --wait WhyCremisi-notarize.zip
          xcrun stapler staple WhyCremisi.dmg

      - name: Code Sign (Windows)
        if: runner.os == 'Windows'
        run: |
          echo "${{ secrets.WIN_CERTIFICATE }}" | base64 --decode > cert.pfx
          signtool sign /fd SHA256 /tr http://timestamp.digicert.com \
            /td SHA256 /f cert.pfx /p "${{ secrets.WIN_CERT_PASSWORD }}" \
            WhyCremisiInstaller.exe

      - name: Package
        run: cmake --build --preset release --target package

      - name: Upload Release Asset
```

```
        with:
          files: dist/*
```

— 8.2 — CHANGELOG GENERATION

The changelog is automatically generated from conventional commits (`feat:`, `fix:`, `chore:`) via `git-cliff`:

```
git-cliff --config cliff.toml --tag "v1.2.0" --output CHANGELOG.md
```

— 8.3 — DRAFT RELEASE PR

Before each release, a `release/vX.Y.Z` PR toward `main` triggers:

1. Staging build on all 3 OS
2. Full test suite execution
3. Release notes draft generation
4. Manual approval required

• 9 — Post-Release Verification

— 9.1 — AUTOMATED SMOKE TEST

```
# Verify code signature (macOS)
codesign --verify --verbose WhyCremisi.vst3
codesign -dvvv WhyCremisi.app

# Verify notarization (macOS)
spctl --assess --verbose=4 --type install WhyCremisi.dmg

# Verify checksum
shasum -a 256 -c WhyCremisi-1.2.0-SHA256SUMS.txt

# Verify Windows signature
signtool verify /pa /v WhyCremisiInstaller.exe
```

— 9.2 — SHA-256 HASH VERIFICATION

```
# Generate checksum
sha256sum dist/* > SHA256SUMS.txt
gpg --detach-sign --armor SHA256SUMS.txt
```

— 9.3 — MALWARE SCAN

Before final release, every build is scanned with:

- **VirusTotal API** (60+ antivirus engines)
- **macOS XProtect** check
- **Windows Defender** scan

```
curl --request POST \
  --url "https://www.virustotal.com/api/v3/files" \
  --header "x-apikey: $VT_API_KEY" \
  --form "file=@WhyCremisi-1.2.0-macOS.dmg"
```

— 9.4 — DAW COMPATIBILITY SMOKE TEST

DAW	PLATFORM	PLUGIN	LOAD	PROCESS	SAVE
Ableton Live 12	macOS/Windows	VST3	✓	✓	✓
Logic Pro 11	macOS	AU	✓	✓	✓
Pro Tools 2024	macOS/Windows	AAX	✓	✓	✓
FL Studio 21	Windows	VST3	✓	✓	✓
Cubase 13	macOS/Windows	VST3	✓	✓	✓
REAPER 7	macOS/Windows/Linux	VST3	✓	✓	✓
Studio One 6	macOS/Windows	VST3	✓	✓	✓

• References

RESOURCE	URL
CMake Presets	https://cmake.org/cmake/help/latest/manual/cmake-presets.7.html
Apple Code Signing	https://developer.apple.com/documentation/security/code_signing
Apple NotaryTool	https://developer.apple.com/documentation/notarytool
Sparkle Framework	https://sparkle-project.org
Squirrel.Windows	https://github.com/Squirrel/Squirrel.Windows
Inno Setup	https://jrsoftware.org/isinfo.php
git-cliff	https://git-cliff.readthedocs.io
VirusTotal API	https://developers.virustotal.com
JUCE Audio Plugin	https://juce.com